

SOAVIZ STUDIO COWORK DIRECTIVES

페이지별 수정 반영사항과 코워크 지시문

작성 기준

OpenAI식 상위 1% AI 제품 개발자 관점

핵심 목표

2분 안에 결과물, 저장, 크레딧 결제 루프 완성

우선순위

P0 매출/온보딩/신뢰, P1 구조 단순화, P2 저장/권리

SOAVIZ Studio Page-Level Cowork Directives

작성일: 2026-05-06

기준 문서: SOAVIZ-studio-current-assessment-2026-05-06.html

작성 기준: OpenAI식 상위 1% AI 제품 개발자 관점의 코워크 지시문

주의: 이 문서는 OpenAI 공식 평가 문서가 아니라, OpenAI식 AI 플랫폼/제품 CEO 관점과 최상위 제품 엔지니어링 기준을 적용한 실행 지시문입니다.

0. 최종 목표

SOAVIZ Studio의 다음 개발 목표는 기능 추가가 아닙니다. 목표는 첫 사용자가 2분 안에 결과물을 만들고, 저장하고, 크레딧 기반 결제 루프까지 자연스럽게 도달하는 단일 제작 루프를 닫는 것입니다.

제품 포지션은 다음 한 문장으로 고정합니다.

AI 영상 제작자를 위한 결과 중심 Creative OS - 시나리오부터 샷리스트, 키이미지, 영상 편집, Export까지 한 번에 끝내는 제작 루프.

1. 코워크 운영 원칙

1.1 개발자의 기본 판단 기준

모든 페이지 수정은 다음 질문을 통과해야 합니다.

1. 사용자가 2분 안에 결과물을 얻는가?
2. 결과물이 저장되고 다시 열리는가?
3. Generate, Key Image, Compare, Export 중 하나 이상이 크레딧 루프와 연결되는가?
4. 실패 시 mock이 아니라 명확한 에러, 재시도, Demo Mode로 분리되는가?
5. 상업용 사용 가능성에 대한 출처/권리 로그가 남는가?

1.2 협업 규칙

역할	책임
Product Lead	첫 루트, 요금제, 크레딧 정책, 페이지 축소 결정을 승인
Frontend Owner	Landing, Sidebar, Onboarding, Generate/Edit UI, empty state 구현
Backend Owner	Credit ledger, Stripe webhook, Supabase/R2 저장 계약 구현
AI Pipeline Owner	Key Image, Video, Voice, Music, SFX 생성 API의 Demo/Live 모드 분리
QA Owner	첫 사용자 루프, 결제 실패/성공, 저장 복구, mock 차단 테스트

1.3 코드 스타일 지시문

Next.js/TypeScript 전환을 기준으로 하되, 단일 HTML 파일 유지가 필요한 부분은 동일한 함수 경계로 구현합니다. 핵심은 UI 이벤트 안에 비즈니스 로직을 섞지 않는 것입니다.

```
type Result<T, E> =  
  | { ok: true; value: T }  
  | { ok: false; error: E };
```

```

type Option<T> =
  | { kind: "some"; value: T }
  | { kind: "none" };

type PlanId = "guest" | "free" | "standard" | "pro";
type RuntimeMode = "demo" | "live";

type CreditAction =
  | "keyImage.generate"
  | "video.generate"
  | "video.compare"
  | "audio.extract"
  | "voice.generate"
  | "export.final";

type CreditQuote = {
  action: CreditAction;
  plan: PlanId;
  cost: number;
  remainingBefore: number;
  remainingAfter: number;
  requiresUpgrade: boolean;
};

type ProvenanceRecord = {
  assetId: string;
  assetType: "image" | "video" | "voice" | "music" | "sfx" | "reference" | "other";
  prompt?: string;
  sourceUrl?: string;
  sourceOwner?: string;
  modelName?: string;
  licenseStatus: "unknown" | "user-confirmed" | "platform-generated" | "blockchain";
  createdAt: string;
};

```

모든 중요한 액션은 다음 4단계 순수 함수 흐름을 따릅니다.

```

const runPaidAction = async <TInput, TOutput>(
  ctx: UserContext,
  action: CreditAction,

```

```
input: TInput,
execute: (input: TInput) => Promise<Result<TOutput, AppError>>
): Promise<Result<TOutput, AppError>> => {
  const quote = quoteCredit(ctx, action);
  if (quote.requiresUpgrade) return fail(upgradeRequired(quote));

  const reserved = await reserveCredit(ctx.userId, quote);
  if (!reserved.ok) return reserved;

  const output = await execute(input);
  if (!output.ok) {
    await refundCredit(ctx.userId, reserved.value.ledgerId);
    return output;
  }

  await commitCredit(ctx.userId, reserved.value.ledgerId);
  return output;
};
```

2. 중요도 순서별 전역 지시문

P0-01. 결제와 크레딧 루프를 먼저 닫는다

적용 페이지: Credits/Pricing, Settings/Billing, Generate, Key Image, Compare, Export

개발 지시문:

1. `stripeUrls.standard`, `stripeUrls.pro`, `checkoutUrl` 을 `null` 상태로 배포하지 않습니다.
2. Payment Link가 없으면 결제 버튼은 비활성화하고, Admin/Settings에서 누락 상태를 표시합니다.
3. Stripe 성공 후 클라이언트 local state만 바꾸지 말고, 서버의 `plan`, `credits`, `ledger` 를 업데이트 합니다.
4. Generate, Key Image, Compare, Export 버튼에는 액션 전 예상 크레딧 차감량을 노출합니다.
5. 크레딧 부족 시 결제 모달은 "업그레이드 권유"가 아니라 "이 액션을 완료하기 위해 필요한 크레딧"을 보여줘야 합니다.

완료 기준:

사용자가 Free 상태에서 Export 클릭

- > 필요 크레딧 표시
- > Upgrade 모달
- > Stripe 이동
- > 성공 복귀
- > plan/credits 갱신
- > Export 재시도 가능

P0-02. 첫 진입 루트를 하나로 고정한다

적용 페이지: Landing, Today, Edit/Video Editor, Create, Pipeline

권장 결정:

Landing

- > 샘플로 체험하기
- > Video Editor 온보딩
- > 15초 샘플 영상 자동 분석

- > A/B/C 버전 제안
- > Export
- > Upgrade

개발 지시문:

1. Landing의 CTA는 하나의 Primary CTA로 통일합니다.
2. `today`, `create`, `pipeline`, `edit` 중 첫 사용자 default route는 하나만 허용합니다.
3. 신규 사용자의 empty state에는 "샘플 프로젝트로 시작"을 넣습니다.
4. 첫 루트 중간에 Settings, Library, Admin, Guide로 흩어지지 않게 합니다.

P0-03. Mock fallback을 Demo Mode로 분리한다

적용 페이지: Key Image, Generate, Video Editor, Audio/Extract

개발 지시문:

1. API 실패 시 mock 결과를 live 결과 카드에 넣지 않습니다.
2. Demo Mode는 UI 상단에 명확한 배지를 표시합니다.
3. Live Mode에서는 API 실패를 에러 상태로 처리하고, 재시도/설정 이동/서버 상태 확인 CTA를 제공합니다.
4. 유료 액션에서는 mock 결과 생성 시 크레딧을 차감하지 않습니다.

함수 계약:

```
type GenerationRequest<TInput> = {
  mode: RuntimeMode;
  input: TInput;
  userId: string;
};

type GenerationOutput<TAsset> =
  | { kind: "live"; asset: TAsset; provenance: ProvenanceRecord }
  | { kind: "demo"; asset: TAsset; demoReason: string }
  | { kind: "failed"; error: AppError };

function assertLiveResult<TAsset>(
  output: GenerationOutput<TAsset>
): Result<Extract<GenerationOutput<TAsset>, { kind: "live" }>, AppError> {
```

```
if (output.kind !== "live") return fail({ code: "LIVE_RESULT_REQUIRED" });
return ok(output);
}
```

P0-04. Admin은 클라이언트 앱에서 분리한다

적용 페이지: Admin, Settings

개발 지시문:

1. 공개 HTML 내부에서 Admin UI가 보이면 안 됩니다.
2. Admin route는 feature flag와 role guard를 모두 통과해야 합니다.
3. 운영 지표는 별도 관리자 URL 또는 서버 권한이 있는 대시보드로 분리합니다.

완료 기준:

```
guest/free/standard/pro 사용자 -> Admin 메뉴 미노출
admin role 사용자 -> 별도 관리자 URL에서 접근
client source에 민감한 운영 액션 미노출
```

P1-01. 사이드바를 9개 이하로 축소한다

추천 구조:

```
Today
Create
Characters
Pre Production
Generate
Edit
Assets
Library
Settings
```

흡수 정책:

현재 메뉴	처리
Voice	Assets 하위 Voice 탭
Music	Assets 하위 Music 탭
SFX	Assets 하위 SFX 탭
Audio	Assets 대표 메뉴
Cinema	Style Reference 탭 또는 Library Reference 탭
Compare	Generate/Edit 내부 모드
Memory Graph	Library 고급 기능, 조건부 노출
Project Archive	Library Export 탭
Admin	별도 관리자 URL

3. 페이지별 수정 반영사항

3.1 Credits / Pricing

중요도: P0

현재 점수: 58

최종 목표: 가격표가 아니라 실제 매출 전환 페이지로 바꿉니다.

수정 지시문:

1. Standard/Pro 플랜의 Stripe Payment Link를 실제 값으로 연결합니다.
2. Free, Standard, Pro의 월 제공 크레딧과 액션별 차감량을 같은 표 안에 표시합니다.
3. 결제 버튼 클릭 전 사용자가 얻는 결과물을 명확히 씁니다. 예: "월 120 크레딧으로 Export 12회".
4. 결제 성공/실패/취소 query state를 처리합니다.
5. 가격표 UI와 실제 billing state가 불일치하면 상단 경고를 표시합니다.

함수 지시문:

```
type CheckoutState =
  | { kind: "idle" }
  | { kind: "redirecting"; plan: Exclude<PlanId, "guest" | "free"> }
  | { kind: "success"; sessionId: string }
  | { kind: "cancelled"; plan?: PlanId }
  | { kind: "failed"; reason: string };

function resolveCheckoutUrl(plan: "standard" | "pro", env: BillingEnv): Resu
function parseCheckoutReturn(search: URLSearchParams): CheckoutState;
function syncBillingState(userId: string, sessionId: string): Promise<Result
```

QA:

Payment Link 없음 -> 버튼 disabled
 결제 취소 -> plan 유지, 재시도 가능
 결제 성공 -> credits 갱신, toast + Billing 카드 반영

3.2 Landing

중요도: P0

현재 점수: 74

최종 목표: 신규 사용자를 하나의 성공 루프로 밀어 넣습니다.

수정 지시문:

1. Primary CTA를 "샘플 영상으로 2분 체험"으로 고정합니다.
2. Secondary CTA는 숨기거나 "가격 보기" 정도로 제한합니다.
3. CTA 클릭 시 `edit?sample=starter` 또는 결정된 onboarding route로 이동합니다.
4. landing에서 제품 기능을 나열하지 말고, 첫 결과물 하나를 약속합니다.
5. hero 문구는 "AI가 만든다. 당신이 완성한다"를 유지하되, supporting copy에 "15초 샘플 영상 분석 -> 3개 버전 제안 -> Export"를 넣습니다.

함수 지시문:

```
type OnboardingEntry = "sample-video" | "create-shotlist";

function selectOnboardingEntry(user: UserContext): OnboardingEntry {
  if (user.hasExistingProject) return "create-shotlist";
  return "sample-video";
}

function buildOnboardingHref(entry: OnboardingEntry): string {
  return entry === "sample-video"
    ? "/edit?sample=starter"
    : "/create?template=story-to-shotlist";
}
```

QA:

```
비로그인 사용자 CTA -> sample route
기존 프로젝트 사용자 CTA -> 이어서 작업 또는 sample 선택
모바일에서 CTA가 첫 화면 안에 보임
```

3.3 Edit / Video Editor

중요도: P0

현재 점수: 86

최종 목표: SOAVIZ의 첫 유료 성공 경험이 되는 메인 온보딩 화면으로 승격합니다.

수정 지시문:

1. 샘플 영상 로드 버튼을 첫 화면 기본 상태로 노출합니다.
2. "자동 분석 -> 버전 제안 -> 미리보기 -> Export"를 stepper로 표시합니다.
3. Export 버튼에는 크레딧 차감량과 워터마크 여부를 표시합니다.
4. 분석 실패 시 Demo Mode 샘플 결과와 Live Mode 에러를 분리합니다.
5. 사용자가 만든 버전은 Library와 Project에 즉시 저장합니다.

함수 지시문:

```
type EditorStep =
  | "sampleLoaded"
  | "analyzing"
  | "versionsReady"
  | "previewReady"
  | "exporting"
  | "exported";

type EditVersion = {
  id: string;
  label: "A" | "B" | "C";
  strategy: "pacing" | "hook" | "clarity";
  changes: string[];
  renderUrl?: string;
  score?: number;
};

function reduceEditorStep(state: EditorState, event: EditorEvent): EditorState {
  function rankEditVersions(versions: EditVersion[]): EditVersion[] {
    function quoteExport(plan: PlanId, version: EditVersion): CreditQuote {

```

QA:

샘플 영상 클릭 -> 분석 결과 표시
 A/B/C 버전 생성 -> 하나 선택 가능
 Export 클릭 -> 크레딧 확인
 크레딧 부족 -> Upgrade 모달
 Export 성공 -> Library 저장

3.4 Generate / Video Studio

중요도: P0

현재 점수: 73

최종 목표: 모델 라우터 컨셉을 실제 비용/성공률/결과물 중심으로 바꿉니다.

수정 지시문:

1. 모델 선택 UI에 예상 비용, 예상 시간, 실패 가능성을 표시합니다.
2. Generate 버튼에 크레딧 차감량을 표시합니다.
3. 생성 전 입력 이미지/오디오 출처 로그를 요구합니다.
4. 실패 시 credit refund 규칙을 명확히 적용합니다.
5. Compare Mode는 Generate 내부의 "3개 버전 생성" 모드로 흡수합니다.

함수 지시문:

```
type VideoModelId = "runway" | "pika" | "luma" | "klimg" | "demo";

type ModelRoute = {
  model: VideoModelId;
  estimatedSeconds: number;
  estimatedCredits: number;
  qualityBias: "speed" | "quality" | "motion" | "style";
};

function routeVideoModel(input: VideoGenerateInput, ctx: UserContext): ModelRoute {
  function validateVideoInputs(input: VideoGenerateInput): Result<VideoGenerateInput> {
    function createVideoJob(route: ModelRoute, input: VideoGenerateInput): Promise<VideoGenerateOutput> {
```

QA:

입력 누락 -> Generate disabled
 เครดิต 부족 -> Upgrade
 API 실패 -> refund
 성공 -> output/provenance 저장

3.5 Key Image Studio

중요도: P0

현재 점수: 70

최종 목표: 캐릭터/IP 신뢰를 만드는 live 생성 도구로 정리합니다.

수정 지시문:

1. `/api/character-key-image/generate` 실패 시 mock 이미지를 결과 카드에 넣지 않습니다.
2. Demo Mode를 켜 경우에만 mock/sample 이미지를 표시합니다.
3. 생성 버튼 옆에 "Live API 연결됨/연결 필요" 상태를 표시합니다.
4. 참조 이미지가 있으면 출처/권리 확인 체크를 요구합니다.
5. 성공 결과는 Character와 Library 양쪽에 저장합니다.

함수 지시문:

```
type KeyImageInput = {
  characterId: string;
  prompt: string;
  referenceImageIds: string[];
  aspectRatio: "1:1" | "9:16" | "16:9";
  mode: RuntimeMode;
};

function validateKeyImageInput(input: KeyImageInput): Result<KeyImageInput, ...> {
function generateKeyImage(input: KeyImageInput): Promise<GenerationOutput<ImageAsset>> {
function attachKeyImageToCharacter(characterId: string, asset: ImageAsset): {
```

QA:

Live API 없음 -> 생성 전 차단
 Demo Mode on -> Demo 배지 노출
 참조 이미지 출처 미입력 -> 생성 차단
 성공 -> character.card와 library에 동시 반영

3.6 Settings / Billing / API Vault

중요도: P0

현재 점수: 65

최종 목표: 결제, 저장, API 상태를 사용자가 이해할 수 있는 신뢰 페이지로 만듭니다.

수정 지시문:

1. Account, Workspace, Storage, Billing, API Status를 탭으로 분리합니다.
2. API 키 입력 모델을 "서버 키 사용"과 "개인 키 사용"으로 명확히 구분합니다.
3. Stripe, Supabase, R2, AI API health를 한 줄 상태로 보여줍니다.
4. Storage policy를 Guest/Signed-in/Paid로 설명합니다.
5. Admin 기능은 Settings에서 제거합니다.

함수 지시문:

```
type ServiceHealth = {
  service: "stripe" | "supabase" | "r2" | "openai" | "video-model";
  status: "ok" | "degraded" | "down" | "not-configured";
  checkedAt: string;
  actionHref?: string;
};

function summarizeHealth(health: ServiceHealth[]): "ready" | "partial" | "blocked";
function resolveStoragePolicy(plan: PlanId): StoragePolicy;
```

QA:

Stripe URL 없음 -> Billing blocked
 R2 없음 -> Paid storage warning

OpenAI key 없음 -> Live generation blocked

Guest user -> local only 설명

3.7 Today

중요도: P1

현재 점수: 78

최종 목표: 내부 대시보드와 신규 사용자 empty state를 분리합니다.

수정 지시문:

1. 신규 사용자에게는 "샘플 영상으로 시작"과 "세계관으로 시작"만 보여줍니다.
2. 기존 사용자에게는 최근 프로젝트, pending render, credit 상태를 보여줍니다.
3. Today가 첫 루트가 아니라면 onboarding 완료 후 dashboard로 사용합니다.
4. 이어서 작업 CTA는 마지막 편집 지점을 복원해야 합니다.

함수 지시문:

```
type TodayState =
  | { kind: "new-user"; recommendedEntry: OnboardingEntry }
  | { kind: "active"; recentProjects: ProjectSummary[]; credit: CreditBalance }
  | { kind: "blocked"; reason: AppError };

function resolveTodayState(ctx: UserContext, projects: ProjectSummary[]): TodayState {
```

3.8 Create / Personas

중요도: P1

현재 점수: 84

최종 목표: 세계관 -> 스토리 -> 시나리오 -> 샷리스트 연결을 제품의 두 번째 강한 루프로 유지합니다.

수정 지시문:

1. Create 결과의 다음 행동을 "샷리스트 생성"으로 고정합니다.
2. TTS/영상 연결 버튼은 보조 행동으로 낮춥니다.

3. 생성된 시나리오는 Pipeline 프로젝트로 자동 변환 가능해야 합니다.
4. 결과 복사보다 "프로젝트로 저장"을 우선합니다.

함수 지시문:

```
type StorySeed = {
  personaId: string;
  world: string;
  conflict: string;
  tone: string;
};

function createScenario(seed: StorySeed): Promise<Result<Scenario, AppError>>
function scenarioToShotlist(scenario: Scenario): Promise<Result<Shot[], AppError>>
function persistScenarioProject(scenario: Scenario, shots: Shot[]): Promise<Result<ScenarioProject, AppError>>
```

3.9 Pre Production / Pipeline

중요도: P1

현재 점수: 80

최종 목표: SOAVIZ의 제작 OS 차별점을 담당하는 샷리스트/보드 화면으로 정리합니다.

수정 지시문:

1. 프로젝트 선택 전 empty state에 sample pipeline을 제공합니다.
2. Shot Detail 독립 페이지는 Pipeline 내부 drawer/modal로 흡수합니다.
3. 각 shot에는 Key Image, Video Generate, Edit로 이어지는 next action을 둡니다.
4. pipeline status는 draft -> ready -> generated -> edited -> exported 로 단순화합니다.

함수 지시문:

```
type ShotStatus = "draft" | "ready" | "generated" | "edited" | "exported";

function nextShotAction(shot: Shot): "complete-brief" | "key-image" | "generate-video" | "edit"
function updateShotStatus(shot: Shot, event: ShotEvent): Shot;
```

3.10 Characters

중요도: P1

현재 점수: 82

최종 목표: IP 제작의 중심 데이터 모델로 유지하되, Key Image와 Voice를 연결합니다.

수정 지시문:

1. 캐릭터 카드에는 readiness score를 표시합니다.
2. 부족한 항목은 "보이스 필요", "키이미지 필요", "권리 로그 필요"처럼 액션으로 보여줍니다.
3. 캐릭터 상세에서 Key Image Studio, Voice 탭으로 이동합니다.
4. 캐릭터 관련 asset provenance를 한 곳에서 볼 수 있어야 합니다.

함수 지시문:

```
type CharacterReadiness = {
  profile: boolean;
  voice: boolean;
  keyImage: boolean;
  rights: boolean;
  score: number;
};

function calculateCharacterReadiness(character: Character, assets: Asset[]):
function getCharacterNextAction(readiness: CharacterReadiness): CharacterAct
```

3.11 Projects

중요도: P1

현재 점수: 68

최종 목표: 빈 프로젝트 보관소가 아니라 샘플과 복구가 있는 시작점으로 만듭니다.

수정 지시문:

1. 프로젝트가 없으면 샘플 프로젝트 1개를 생성할 수 있게 합니다.
2. 프로젝트 카드에는 마지막 단계, 다음 액션, 저장 위치를 표시합니다.

3. local project와 cloud project를 UI에서 구분합니다.
4. 삭제보다 archive를 기본 행동으로 둡니다.

함수 지시문:

```
function createSampleProject(template: "video-editor" | "story-world"): Prom.  
function resolveProjectStorageBadge(project: Project): "local" | "cloud" | "
```

3.12 Assets: Audio / Voice / Music / SFX

중요도: P1

현재 점수: Audio 78, Voice 75, Music 70, SFX 67, Voices 62, Clean 60

최종 목표: 분산된 오디오 관련 메뉴를 Assets 하나로 통합합니다.

수정 지시문:

1. Audio를 Assets의 대표 메뉴로 승격합니다.
2. Voice, Music, SFX, Extract, Clean은 Assets 내부 탭으로 통합합니다.
3. 오디오에서 이미 구현된 권리 확인 구조를 모든 asset에 재사용합니다.
4. Voice clone 여부와 상업 사용 가능 여부를 반드시 기록합니다.
5. Music/SFX는 MVP 메인 루프 밖의 보조 기능으로 배치합니다.

함수 지시문:

```
type AssetTab = "all" | "voice" | "music" | "sfx" | "extract" | "clean";  
  
function filterAssets(assets: Asset[], tab: AssetTab): Asset[];  
function requireCommercialRights(asset: Asset): Result<Asset, AppError>;  
function createAssetProvenance(asset: Asset, input: ProvenanceInput): Proven
```

QA:

Voice/Music/SFX 메뉴 -> Sidebar에서 제거
Assets 탭 이동 -> 기존 기능 접근 가능

상업 권리 미확인 asset -> Export 차단 또는 warning

3.13 Library

중요도: P2

현재 점수: 66

최종 목표: 결과 보관소에서 재사용/채택/export 루프로 바꿉니다.

수정 지시문:

1. asset card에 "Use in Generate", "Use in Edit", "Export"를 표시합니다.
2. empty state는 "첫 결과물을 만들러 가기"로 연결합니다.
3. Memory Graph는 데이터 20개 이상 또는 feature flag에서만 노출합니다.
4. Project Archive는 Library의 Export 탭으로 흡수합니다.

함수 지시문:

```
function suggestAssetReuse(asset: Asset): AssetReuseAction[];
function shouldShowMemoryGraph(stats: WorkspaceStats): boolean {
  return stats.acceptedAssets >= 20 && stats.featureFlags.memoryGraph;
}
```

3.14 Style / Cinema Library

중요도: P2

현재 점수: Style 69, Cinema 65

최종 목표: 생성 프롬프트에 실제 영향을 주는 Reference 시스템으로 정리합니다.

수정 지시문:

1. Cinema Library는 독립 메뉴가 아니라 Style의 Reference 탭으로 흡수합니다.
2. Reference 추가 시 URL, 원작자, 스타일 추출 여부, 저작권 고지를 기록합니다.
3. Style Bible은 Generate/Key Image prompt에 자동 주입되는 preview를 보여줍니다.

함수 지시문:

```

type StyleBible = {
  palette: string[];
  cameraLanguage: string[];
  lighting: string[];
  negativePrompts: string[];
  references: ProvenanceRecord[];
};

function compileStyleBible(inputs: StyleInput[]): Result<StyleBible, AppError>
function injectStyleIntoPrompt(prompt: string, bible: StyleBible): string;

```

3.15 Compare Mode

중요도: P2

현재 점수: 77

최종 목표: 단독 페이지가 아니라 Generate/Edit의 고급 모드로 흡수합니다.

수정 지시문:

1. Sidebar에서 Compare를 제거합니다.
2. Generate에는 "3 Variants" toggle로 넣습니다.
3. Edit에는 A/B/C 버전 비교 panel로 넣습니다.
4. 베스트 채택 시 Library와 Project history에 decision log를 남깁니다.

함수 지시문:

```

type CompareDecision = {
  variants: EditVersion[];
  selectedId: string;
  reason: string;
  decidedAt: string;
};

function scoreVariants(variants: EditVersion[]): EditVersion[];
function recordCompareDecision(projectId: string, decision: CompareDecision)

```

3.16 Story View

중요도: P2

현재 점수: 63

최종 목표: 독립 페이지보다 Create 내부 결과 상태로 흡수합니다.

수정 지시문:

1. Story View route는 legacy alias로만 유지합니다.
2. 신규 UI에서는 Create 결과 panel로 표시합니다.
3. 다음 액션은 Copy보다 "Shotlist 생성"과 "Project 저장"을 우선합니다.

3.17 History / Production Log

중요도: P2

현재 점수: 64

최종 목표: 사용자의 제작 신뢰를 위한 자동 기록으로 유지하되, 메인 메뉴에서는 숨깁니다.

수정 지시문:

1. Today 하위 "Recent Activity" 또는 Library 하위 "Production Log"로 이동합니다.
2. credit ledger, provenance, export history와 연결합니다.
3. 사람이 직접 쓰는 로그가 아니라 자동 이벤트 로그를 기본으로 합니다.

3.18 Project Archive

중요도: P3

현재 점수: 56

최종 목표: 독립 페이지를 제거하고 Library Export 탭으로 흡수합니다.

수정 지시문:

1. Sidebar에서 제거합니다.
2. 완료 프로젝트는 Library의 Export/Archive tab에 표시합니다.
3. archive 상태에서도 원본 asset과 provenance는 유지합니다.

3.19 Memory Graph

중요도: P3

현재 점수: 60

최종 목표: 데이터가 쌓인 후 드러나는 고급 기능으로 숨깁니다.

수정 지시문:

1. 기본 Sidebar에서 제거합니다.
2. accepted asset 20개 이상 또는 admin feature flag에서만 노출합니다.
3. 그래프가 없을 때는 placeholder를 보여주지 않습니다.

3.20 Shot Detail

중요도: P3

현재 점수: 55

최종 목표: 독립 페이지를 없애고 Pipeline 내부 상세 패널로 흡수합니다.

수정 지시문:

1. `/shot/:id` 독립 진입은 legacy route로 유지합니다.
2. Pipeline board에서 drawer로 열리게 합니다.
3. Shot 안에서 Key Image, Generate, Edit next action을 바로 실행합니다.

3.21 Admin

중요도: P0 Security

현재 점수: 45

최종 목표: 공개 클라이언트에서 제거합니다.

수정 지시문:

1. 일반 사용자 bundle에서 Admin 메뉴와 화면을 숨깁니다.
2. 관리자 기능은 서버 권한을 확인하는 별도 경로로 이동합니다.

3. client HTML 안에 운영 지표 mutation 코드가 있으면 제거합니다.

4. 라우팅과 메뉴 정리 지시문

현재 alias는 사용자를 살리기 위한 호환 계층으로만 유지합니다. 신규 제품 구조는 다음 단일 route map으로 압축합니다.

```
type MainRoute =  
  | "today"  
  | "create"  
  | "characters"  
  | "pipeline"  
  | "generate"  
  | "edit"  
  | "assets"  
  | "library"  
  | "settings";  
  
const LEGACY_ROUTE_ALIASES: Record<string, MainRoute> = {  
  "voice": "assets",  
  "voices": "assets",  
  "music": "assets",  
  "sfx": "assets",  
  "audio": "assets",  
  "extract": "assets",  
  "clean": "assets",  
  "compare": "generate",  
  "cinema": "library",  
  "story": "create",  
  "shot": "pipeline",  
  "archive": "library"  
};  
  
function resolveRoute(rawRoute: string): MainRoute {  
  return isMainRoute(rawRoute) ? rawRoute : LEGACY_ROUTE_ALIASES[rawRoute] ?  
}
```

완료 기준:

Sidebar visible items ≤ 9

Legacy URL 접근 가능

신규 사용자 default route는 하나

Admin은 일반 사용자에게 미노출

5. 저장 구조 지시문

저장은 기능이 아니라 신뢰의 기반입니다. 다음 정책을 모든 페이지에 적용합니다.

```
Guest Mode: localStorage / IndexedDB
Signed-in Mode: Supabase projects, assets, outputs
Paid Mode: R2 storage + credit ledger + export history
```

함수 계약:

```
type StoragePolicy = {
  mode: "local" | "cloud" | "r2";
  canSync: boolean;
  canExportHistory: boolean;
  maxAssetBytes?: number;
};

function resolveStoragePolicy(plan: PlanId): StoragePolicy {
  switch (plan) {
    case "guest":
    case "free":
      return { mode: "local", canSync: false, canExportHistory: false };
    case "standard":
      return { mode: "cloud", canSync: true, canExportHistory: true };
    case "pro":
      return { mode: "r2", canSync: true, canExportHistory: true };
  }
}

async function persistAsset(asset: Asset, ctx: UserContext): Promise<Result<
  const policy = resolveStoragePolicy(ctx.plan);
  return matchStorage(policy, {
    local: () => saveToIndexedDb(asset),
    cloud: () => saveToSupabase(asset),
    r2: () => saveToR2(asset)
```

```
});  
}
```

6. 권리 로그 지시문

상업용 사용 가능성은 Export 직전에 갑자기 묻지 않습니다. 생성/업로드/참조 단계부터 누적합니다.

```
function requireProvenanceForExport(assets: Asset[]): Result<ProvenanceRecord> {
  const missing = assets.filter((asset) => !asset.provenance || asset.provenance.isMissing())
  if (missing.length > 0) {
    return fail({
      code: "PROVENANCE_REQUIRED",
      message: "Export 전에 모든 사용 asset의 출처/권리 로그가 필요합니다.",
      details: { assetIds: missing.map((asset) => asset.id) }
    });
  }
  return ok(assets.map((asset) => asset.provenance));
}
```

적용 대상:

대상	필수 기록
Key Image	prompt, reference source, model, createdAt
Video	model, input image/audio source, license
Voice	owner, clone 여부, commercial use
Music/SFX	source, generator, usage purpose
Cinema Reference	URL, author, extraction purpose, copyright notice
Export	final asset manifest, credit ledger id, provenance bundle

7. QA 실행 시나리오

7.1 첫 사용자 수익 루프

1. file/app 진입
2. Landing CTA 클릭
3. 샘플 Video Editor 진입
4. 자동 분석 완료
5. A/B/C 버전 제안
6. 베스트 선택
7. Export 클릭
8. 크레딧 부족 확인
9. Upgrade modal
10. Stripe 성공 복귀
11. Export 성공
12. Library 저장 확인

7.2 Mock 차단 루프

1. Live Mode에서 Key Image API 끄기
2. Generate 클릭
3. mock 이미지가 결과 카드에 나오지 않아야 함
4. 에러 메시지와 Settings 이동 CTA 표시
5. Demo Mode 켜기
6. Demo 배지와 sample 결과 표시

7.3 저장 복구 루프

1. Guest로 sample project 생성
2. 새로고침
3. IndexedDB/localStorage에서 복구
4. 로그인 전환
5. cloud sync 안내

6. paid 전환

7. R2/export history 활성화

8. 개발 순서

Sprint 1: P0 Revenue and Onboarding

1. Credits/Pricing Payment Link 연결
2. Billing return state 처리
3. Landing CTA 단일화
4. Video Editor sample onboarding
5. Export credit quote 표시
6. Admin 숨김

Sprint 2: P0 Trust and Live Mode

1. Key Image mock fallback 제거
2. Demo Mode 명시
3. Generate model router cost/credit 표시
4. API health 상태와 Settings 연결
5. Credit reserve/refund/commit 흐름 추가

Sprint 3: P1 Product Simplification

1. Sidebar 9개 이하로 축소
2. Assets 통합
3. Compare를 Generate/Edit로 흡수
4. Story View/Shot Detail legacy 처리
5. Project empty state와 sample project 추가

Sprint 4: P2 Persistence and Rights

1. Guest/Signed-in/Paid 저장 정책 반영
2. Provenance schema 추가
3. Export manifest 생성
4. Library reuse actions 추가

5. Memory Graph 조건부 노출

9. Definition of Done

P0 완료 조건:

- 사용자가 Landing에서 시작해 Export/Upgrade까지 끊기지 않는다.
- 결제 URL이 null인 상태로 배포되지 않는다.
- Live Mode에서 mock 결과가 유료 결과처럼 보이지 않는다.
- Admin은 일반 사용자에게 보이지 않는다.

P1 완료 조건:

- Sidebar가 9개 이하이다.
- Assets가 Voice/Music/SFX/Audio를 흡수한다.
- Create/Pipeline/Characters가 하나의 IP 제작 루프로 이어진다.

P2 완료 조건:

- 모든 export asset에 provenance bundle이 붙는다.
 - Paid 사용자의 결과물은 R2/export history와 연결된다.
 - Library에서 채택/재사용/export 흐름이 가능하다.
-

10. 최종 코워크 메시지

개발팀은 "페이지를 더 만들기"보다 "첫 성공 루프를 닫기"에 집중해야 합니다. SOAVIZ Studio는 이미 화면 수와 기능 수가 충분합니다. 지금 필요한 것은 OpenAI식 AI 제품 개발 기준에서 가장 강한 단일 결과물, 즉 **2분 안**에 생성되고, 저장되고, 결제까지 이어지는 영상 제작 경험입니다.

이번 수정의 핵심은 다음과 같습니다.

1. Credits/Pricing과 Export를 연결한다.
2. Landing과 Video Editor를 하나의 온보딩 루프로 묶는다.
3. Key Image와 Generate의 mock을 Demo Mode로 분리한다.
4. Sidebar를 줄이고 Assets/Library로 흡수한다.
5. 저장/권리/크레딧 로그를 모든 결과물에 붙인다.

SOAVIZ Studio Page-Level Cowork Directives · Generated for local review